Respuestas del Cuestionario

- 1. ¿Qué significa la palabra final en la declaración de un atributo? Significa que el valor de ese atributo no puede ser modificado una vez que ha sido inicializado.
- 2. ¿Cómo se representa en UML un atributo final? Se suele escribir el nombre del atributo en mayúsculas sostenidas (por ejemplo, PREFIJO_CODIGO) o indicando su naturaleza de solo lectura.
- 3. ¿Por qué los objetos de tipo Piso se construyen o instancian dentro de la clase Edificio? Porque representan una relación de composición (relación TODO/PARTE), donde el ciclo de vida de la parte (Piso) es controlado de manera exclusiva y total por el todo (Edificio).
- 4. ¿Por qué el método buscarPiso de la clase Edificio DEBE ser privado? Porque forma parte de la lógica interna de encapsulamiento del edificio; los objetos externos no deben tener acceso directo ni manipular la estructura interna de almacenamiento de los pisos.
- 5. Suponga que existe un objeto de tipo Edificio llamado e1... ¿Qué pasa con los objetos de tipo Piso si e1 apunta a null? Al romperse la única referencia al objeto Edificio, los objetos Piso asociados quedan inaccesibles en memoria y son eliminados, cumpliendo con el control estricto del ciclo de vida de la composición.
- 6. ¿Qué opinión tiene usted sobre el diagrama de actividad presentado? Es una herramienta clara y sumamente útil para evaluar de forma metódica qué tipo de relación estructural o no estructural aplicar según las características de contención y dependencia del contexto.
- 7. Describa con sus propias palabras qué es una relación de:
    - Asociación: Relación estructural bidireccional o unidireccional donde dos objetos colaboran y mantienen referencias mutuas o unilaterales sin que uno sea dueño absoluto del ciclo de vida del otro.
    - Agregación: Relación estructural de tipo TODO/PARTE de pertenencia débil, donde la parte puede existir de forma independiente y su ciclo de vida no depende totalmente del todo.
    - Composición: Relación estructural de tipo TODO/PARTE de pertenencia fuerte, donde el todo controla totalmente la creación y destrucción de la parte.
    - Dependencia: Relación no estructural temporal donde una clase requiere de otra (pasada por parámetro o método) para cumplir con una responsabilidad puntual, sin almacenar referencias internas permanentes.

Ejemplo 1: Sistema de Gestión Hospitalaria

En el contexto de un sistema automatizado para la gestión de un hospital, se pueden apreciar claramente los cuatro tipos de relaciones entre sus clases principales:

- **a) Asociación (entre Medico y Paciente):** Se establece una relación de asociación bidireccional o unidireccional entre las clases Médico y Paciente. Esta relación es de colaboración mutua donde un médico atiende a múltiples pacientes y un paciente puede ser atendido por varios especialistas a lo largo del tiempo. Ninguno de los dos objetos posee el

control estricto sobre el ciclo de vida del otro; si un paciente es dado de alta o un médico deja el hospital, la otra entidad continúa existiendo de forma totalmente independiente.

- **b) Agregación (entre DepartamentoMedico y Medico):** Existe una relación de tipo TODO/PARTE de pertenencia débil entre la clase contenedora DepartamentoMedico y los objetos Medico que lo conforman. Un departamento agrupa a un conjunto de médicos para operar de manera coordinada; sin embargo, si el departamento médico llegara a disolverse o cerrarse, los médicos no desaparecen del sistema, sino que pueden ser reasignados a otro departamento, lo que demuestra que la existencia de la parte no depende de la existencia del todo.

- **c) Composición (entre ExpedienteClinico y SeccionHistorial):** Se implementa una relación de tipo TODO/PARTE de pertenencia fuerte y exclusiva entre la clase ExpedienteClinico y las instancias de SeccionHistorial (tales como diagnósticos, recetas o notas de evolución). Las secciones del historial clínico son creadas y gestionadas internamente por el expediente, de modo que no tienen sentido ni existencia fuera de él. Si el expediente clínico es destruido o eliminado del sistema, todas sus secciones asociadas son eliminadas automáticamente de la memoria.

- **d) Dependencia (entre SistemaReceta y Farmacia):** Se establece una relación de dependencia temporal cuando la clase SistemaReceta requiere enviar de manera transitoria un objeto de tipo Farmacia como parámetro en uno de sus métodos (por ejemplo, procesarDespacho(Farmacia farmaciaExterna)) para validar la disponibilidad de medicamentos. La clase SistemaReceta utiliza a la Farmacia en un momento puntual para cumplir con su responsabilidad, pero no almacena ninguna referencia permanente hacia ella como atributo interno.

Ejemplo 2: Sistema de Gestión Académica Universitaria

Dentro de la arquitectura de software para la administración de una institución educativa, las interacciones entre los componentes se estructuran mediante las siguientes relaciones:

- **a) Asociación (entre Estudiante y Curso):** Se define una relación de asociación entre las clases Estudiante y Curso, permitiendo que un estudiante se matricule en varias asignaturas y un curso esté compuesto por múltiples alumnos. Esta relación colaborativa permite el intercambio de calificaciones y asistencias, manteniendo ambas entidades sus ciclos de vida separados; si un estudiante cancela un curso, este último sigue existiendo para el resto del estudiantado.

- **b) Agregación (entre Escuela y Laboratorio):** Se presenta una relación de agregación entre la clase Escuela (como entidad contenedora) y los objetos de tipo Laboratorio que administra. Una escuela agrupa varios laboratorios informáticos para impartir lecciones prácticas. No obstante, si la escuela sufre una reestructuración organizativa y desaparece, los laboratorios físicos e instalaciones tecnológicas se conservan y pueden ser asignados a otra división académica.

- **c) Composición (entre PlanDeEstudio y CursoPlanificado):** Se establece una relación de composición estricta entre un PlanDeEstudio y los bloques o CursoPlanificado que lo constituyen. Un plan de estudios es el responsable absoluto de instanciar y administrar internamente los cursos que lo componen según el año correspondiente. Si el plan de estudios

se actualiza de forma radical o se elimina, los cursos que pertenecían exclusivamente a su estructura interna quedan obsoletos y son removidos del contexto.

- **d) Dependencia (entre GeneradorReportes y ImpresoraPDF):** Se genera una relación de dependencia cuando la clase GeneradorReportes invoca un método que recibe temporalmente un objeto de tipo ImpresoraPDF por parámetro (por ejemplo, exportarReporte(ImpresoraPDF impresora)). El generador necesita de la impresora de manera transitoria únicamente para transformar el reporte en un documento físico o digital, sin conservar atributos permanentes de esta herramienta en su estructura de clases.

Respuestas a la Sección L (PrjCasetaPeaje):

- **¿Por qué en System.out.println(a); fue innecesario escribir a.toString()?**

  Porque la clase PrintStream de Java invoca automáticamente el método toString() sobre cualquier objeto que se le pase como argumento en println().

- **¿Qué pasa con el objeto de tipo Vehiculo que llega al método cobrarPeaje después de la ejecución?**

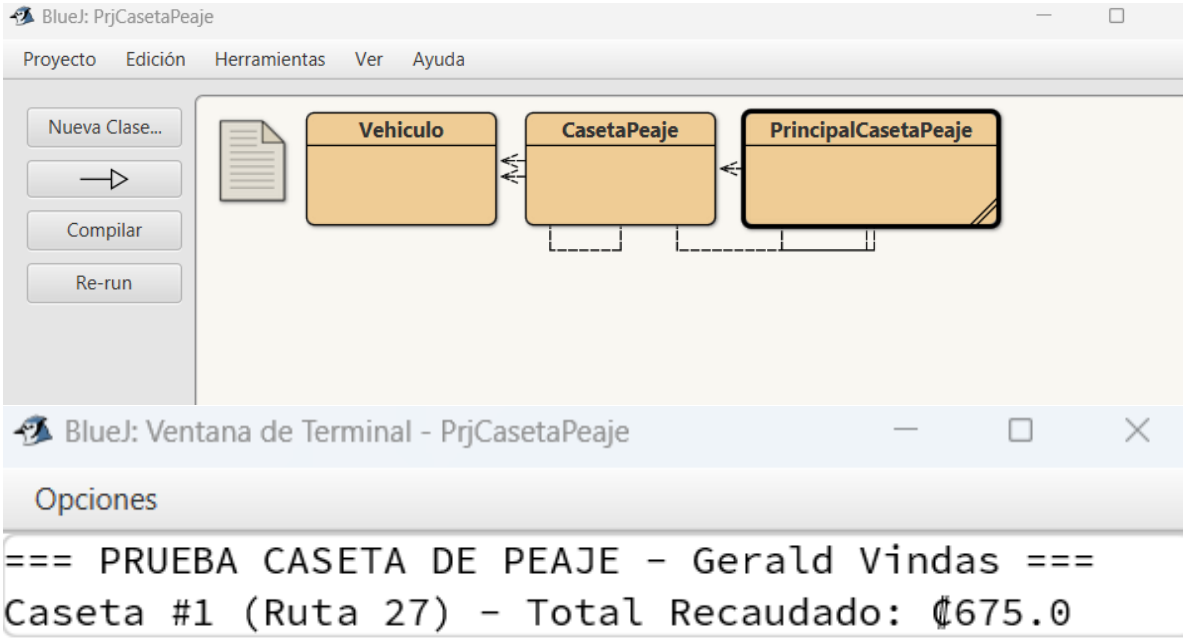
  El objeto sigue existiendo intacto en memoria. Como la relación es de dependencia, la caseta de peaje solo recibe la referencia del vehículo temporalmente para consultar su tipo y calcular el cobro, sin guardarlo en ningún atributo interno.

- **¿Por qué es posible enviar el mensaje obj.getTipo() dentro de cobrarPeaje?**

  Porque obj es una instancia formal de la clase Vehiculo, y getTipo() es un método público perteneciente a la interfaz pública de dicha clase.

- **Explicación de la mejora del encapsulamiento:**

  Se definieron atributos/constantes privadas para las tarifas (TARIFA_PARTICULAR, TARIFA_CARGA_LIVIANA, etc.) dentro de CasetaPeaje. Con esto se eliminaron los valores "quemados" (números mágicos) en el switch de cobrarPeaje, mejorando la mantenibilidad sin alterar la firma del método.



Respuestas de la Sección X (Parte 3: PrjTransporte)

1. **¿Qué tipo de relación estableció entre Ruta y Parada?**

   Se estableció una relación de agregación, ya que las paradas existen de manera independiente a las rutas, se registran previamente y pueden pertenecer a múltiples rutas sin que la ruta controle el ciclo de vida absoluto de las paradas.

2. **¿Cómo representó los horarios de salida y qué relación estableció con Ruta?**

   Se representaron mediante la clase auxiliar HorarioSalida y se enlazaron con una relación de composición, dado que un horario de salida no tiene sentido ni existencia fuera de la ruta que lo agrupa.

3. **¿Qué tipo de relación estableció entre Bus y Ruta?**

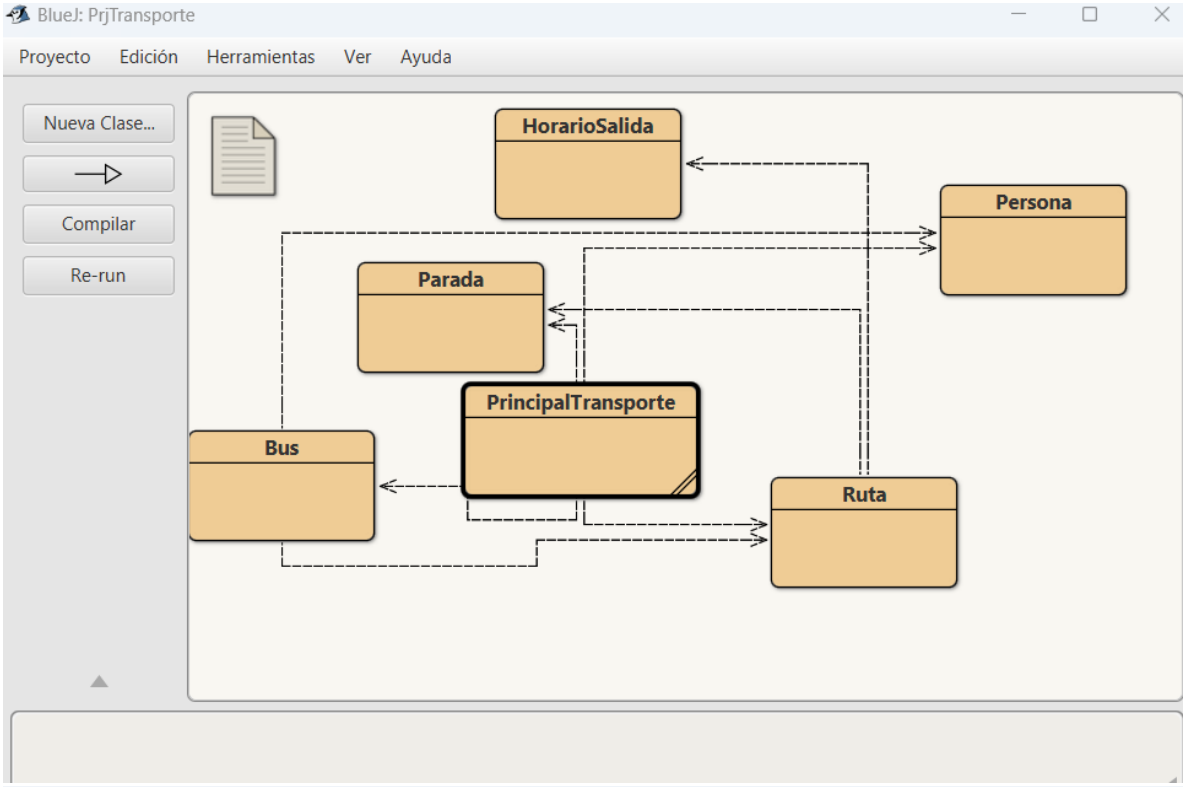
   Una relación de asociación unidireccional, ya que un bus conoce la ruta que cubre temporalmente y esta puede ser asignada o cambiada, mientras que la ruta no requiere almacenar colecciones permanentes de buses.

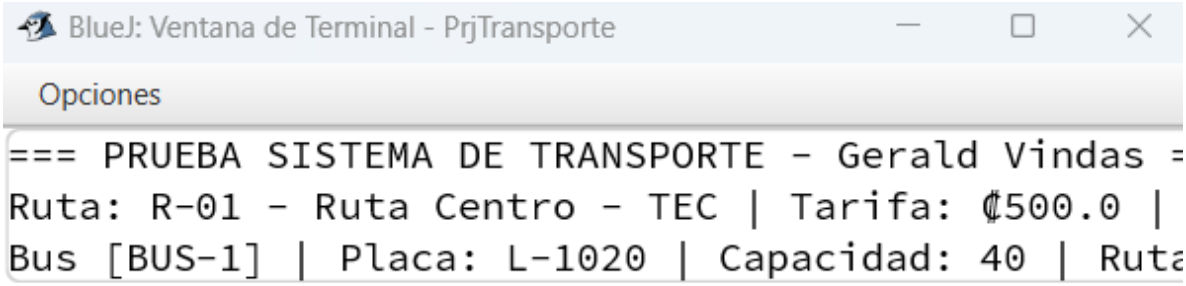
4. **¿Qué tipo de relación estableció entre Bus y Persona?**

   Una relación de dependencia, ya que la persona se recibe temporalmente por parámetro en el método cobrarPasaje(Persona p) del bus únicamente para calcular la tarifa y aplicar descuentos (estudiante o ciudadano de oro) sin almacenar referencias permanentes.

5. **Justificación del diseño y buenas prácticas:**

   Se incorporaron atributos de clase (cantidadBuses) y constantes estáticas (PREFIJO_BUS), además de aplicar sobrecarga de métodos en registrarSalida y en los constructores de Persona para reutilizar código de manera eficiente y robusta.


BlueJ: PrjTransporte
Proyecto
Edición
Herramientas
Ver
Ayuda
Nueva Clase...
Compilar
Re-run
HorarioSalida
Persona
Parada
PrincipalTransporte
Bus
Ruta


BlueJ: Ventana de Terminal - PrjTransporte
Opciones
=== PRUEBA SISTEMA DE TRANSPORTE - Gerald Vindas =
Ruta: R-01 - Ruta Centro - TEC | Tarifa: ₡500.0 |
Bus [BUS-1] | Placa: L-1020 | Capacidad: 40 | Ruta